

1. Narrativa de Elicitación — Historia del Sistema

La siguiente narrativa describe el proceso de elicitación de requisitos tal como lo realizaría un auténtico.

Capítulo I — El primer contacto

Era un martes por la mañana cuando la Dra. Valentina Ramos, directora de la Clínica Veterinaria 'Patitas Felices', llamó a la consultora de software. Llevaba tres años anotando los turnos en una libreta, y dos semanas atrás había perdido los datos de doce clientes porque alguien derramó café sobre el cuaderno.



—Necesito un sistema —dijo, directa—. Algo que me permita registrar a mis clientes, a sus mascotas y los turnos con cada veterinario. Nada complicado.

—Nada complicado —repitió el analista, Marcos Ibáñez, anotando en silencio que 'nada complicado' casi siempre significaba lo contrario.

Marcos coordinó una primera reunión de relevamiento con Valentina y los tres veterinarios de la clínica: el Dr. Leandro Pérez (clínica general), la Dra. Sofía Méndez (dermatología) y el Dr. Tomás Herrera (cirugía). También invitó a Carmen, la recepcionista, quien era —como descubriría Marcos— quien realmente sabía cómo funcionaba la clínica.

Capítulo II — La sesión de relevamiento

—Carmen, ¿cómo manejas los turnos hoy? —preguntó Marcos.

—Tengo la libreta y un Excel en mi computadora —respondió Carmen—. Cuando llaman, anoto el dueño, la mascota y el veterinario disponible. Pero no sé cuándo está cada uno, así que primero tengo que llamar al veterinario por WhatsApp para confirmar.



—¿Cuántos turnos por día?

—Entre veinte y treinta. Los viernes son un caos. A veces se nos superponen y el cliente llega y el veterinario no está.

El Dr. Pérez intervino: —Lo que más necesito yo es ver el historial de la mascota antes de la

consulta. Saber qué tuvo, qué medicación le dieron. Ahora tengo todo en carpetas de papel y a veces no encuentro nada.

La Dra. Méndez asintió: —Y yo necesito saber si el veterinario anterior ya la revisó, para no repetir estudios. Perdemos tiempo y el cliente paga de más.

Marcos anotó todo. Cuando terminó la sesión, tenía cuatro páginas de notas y una certeza: esto no era un sistema de turnos. Era un sistema de gestión integral de una clínica, con múltiples actores, flujos concurrentes y necesidad de historial médico accesible en tiempo real.

—¿Cuántos usuarios simultáneos esperan en el peor caso? —preguntó antes de irse.

—Los tres veterinarios, Carmen y yo —dijo Valentina—. Cinco, quizás seis si contamos a la asistente nueva.

Marcos sonrió. Ese era el número que necesitaba para dimensionar el sistema.

Capítulo III — El análisis y la decisión de arquitectura

De regreso en la consultora, Marcos presentó sus hallazgos al Ingeniero de Software del equipo.

—El sistema tiene cuatro entidades centrales —explicó—: el Dueño, la Mascota, el Veterinario y el Turno. Las relaciones son claras: un dueño puede tener varias mascotas, cada mascota puede tener muchos turnos, y cada turno involucra a un veterinario específico.



—¿Qué volumen de datos esperamos? —preguntó el ingeniero.

—Hoy tienen unos ochocientos clientes activos y cerca de dos mil mascotas registradas. Proyectan crecer un cuarenta por ciento en dos años si el sistema funciona bien.

—Arrancamos con un monolito bien estructurado —decidió el ingeniero—. Lo más importante es que funcione y sea mantenible. Si crecen, migramos a microservicios. Esa migración la vamos a planificar desde el inicio para que no duela cuando llegue el momento.

Esa decisión fue el inicio del documento que el lector tiene en sus manos.

2. Alcance del Sistema

Campo	Detalle
Nombre del sistema	VetSystem — Sistema de Gestión de Clínica Veterinaria
Versión del documento	1.0
Estándar aplicado	IEEE 29148:2018 — Systems and Software Engineering — Requirements Engineering
Cliente / Stakeholder	Clínica Veterinaria 'Patitas Felices'
Analista funcional	Ing. Jorge Agustín Pereyra
Contexto académico	Materia Electiva · Microservicios y APIs Escalables · Universidad de Palermo · 2026

2.1 Descripción general del sistema

VetSystem es un sistema web de gestión integral para clínicas veterinarias. Permite registrar dueños, mascotas, veterinarios y turnos; gestionar el historial médico de cada mascota; y controlar la disponibilidad de veterinarios. El sistema se construye en dos fases: primero como monolito MVC y luego como arquitectura de microservicios.

2.2 Usuarios del sistema

Rol	Descripción	Permisos
ADMIN	Director/a de la clínica — gestión completa del sistema	Acceso total: ABM de todas las entidades + reportes
VETERINARIO	Profesional de la clínica — consulta y gestión de sus turnos	Ver y actualizar sus turnos · ver historial de mascotas
RECEPCIONISTA	Gestión de dueños, mascotas y agenda de turnos	ABM Dueños · ABM Mascotas · Gestión de Turnos

3. Requisitos Funcionales — IEEE 29148

Los siguientes requisitos funcionales fueron identificados durante el proceso de elicitación. Cada requisito sigue la estructura: Identificador · Nombre · Descripción · Prioridad (Alta / Media / Baja) · Criterio de aceptación.

3.1 Gestión de Dueños

RF-01 — Registrar dueño

Descripción: El sistema debe permitir registrar un nuevo dueño con nombre, apellido, DNI, teléfono y email. El DNI debe ser único en el sistema.

Prioridad: Alta

Criterio de aceptación: Dado un DNI no existente, cuando se envía el formulario completo, el sistema devuelve HTTP 201 y el dueño queda persistido en la base de datos.

RF-02 — Consultar dueño

Descripción: El sistema debe permitir consultar los datos de un dueño por ID y listar todos los dueños con paginación.

Prioridad: Alta

Criterio de aceptación: GET /api/duenos/{id} retorna HTTP 200 con los datos del dueño. GET /api/duenos retorna HTTP 200 con lista paginada.

RF-03 — Actualizar dueño

Descripción: El sistema debe permitir actualizar los datos de un dueño existente excepto el DNI.

Prioridad: Media

Criterio de aceptación: PUT /api/duenos/{id} con datos válidos retorna HTTP 200 con los datos actualizados.

RF-04 — Eliminar dueño

Descripción: El sistema debe permitir eliminar un dueño. Si el dueño tiene mascotas con turnos futuros, el sistema debe advertir antes de eliminar.

Prioridad: Media

Criterio de aceptación: DELETE /api/duenos/{id} retorna HTTP 204 si no tiene turnos futuros. Retorna HTTP 409 con mensaje descriptivo si los tiene.

3.2 Gestión de Mascotas

RF-05 — Registrar mascota

Descripción: El sistema debe permitir registrar una mascota asociada a un dueño existente, con nombre, especie, raza y fecha de nacimiento.

Prioridad: Alta

Criterio de aceptación: Dado un dueño existente, cuando se registra la mascota con datos completos, el sistema devuelve HTTP 201 y la mascota queda asociada al dueño.

RF-06 — Consultar mascotas de un dueño

Descripción: El sistema debe permitir listar todas las mascotas asociadas a un dueño específico.

Prioridad: Alta

Criterio de aceptación: GET /api/duenos/{id}/mascotas retorna HTTP 200 con la lista de mascotas del dueño.

RF-07 — Actualizar mascota

Descripción: El sistema debe permitir actualizar los datos de una mascota existente.

Prioridad: Media

Criterio de aceptación: PUT /api/mascotas/{id} con datos válidos retorna HTTP 200 con los datos actualizados.

3.3 Gestión de Veterinarios

RF-08 — Registrar veterinario

Descripción: El sistema debe permitir registrar un veterinario con nombre, apellido, número de matrícula y especialidad. La matrícula debe ser única.

Prioridad: Alta

Criterio de aceptación: POST /api/veterinarios con datos válidos retorna HTTP 201. Un segundo registro con la misma matrícula retorna HTTP 409.

RF-09 — Consultar veterinarios disponibles

Descripción: El sistema debe permitir consultar la lista de veterinarios disponibles. Esta consulta debe estar cacheada para reducir la carga en el microservicio.

Prioridad: Alta

Criterio de aceptación: GET /api/veterinarios retorna HTTP 200 con lista. En la fase de microservicios, la segunda consulta dentro del TTL de 60 segundos no genera una nueva consulta a la base de datos (verificable via logs).

3.4 Gestión de Turnos

RF-10 — Registrar turno

Descripción: El sistema debe permitir registrar un turno con fecha, hora, motivo, mascota y veterinario. No deben existir dos turnos para el mismo veterinario en la misma fecha y hora.

Prioridad: Alta

Criterio de aceptación: POST /api/turnos con datos válidos retorna HTTP 201. Un segundo turno con mismo veterinario, fecha y hora retorna HTTP 409 con mensaje descriptivo.

RF-11 — Consultar turnos por veterinario

Descripción: El sistema debe permitir listar los turnos asignados a un veterinario específico, filtrados por fecha.

Prioridad: Alta

Criterio de aceptación: GET /api/turnos?veterinarioId={id}&fecha={fecha} retorna HTTP 200 con los turnos del veterinario en esa fecha.

RF-12 — Actualizar estado de turno

Descripción: El sistema debe permitir actualizar el estado de un turno (PENDIENTE, EN_CURSO, FINALIZADO, CANCELADO) y agregar observaciones.

Prioridad: Alta

Criterio de aceptación: PUT /api/turnos/{id}/estado con estado válido retorna HTTP 200. Una transición inválida (ej: FINALIZADO → PENDIENTE) retorna HTTP 422.

RF-13 — Consultar historial de turnos de una mascota

Descripción: El sistema debe permitir consultar el historial completo de turnos de una mascota, incluyendo observaciones de cada consulta. En la fase de microservicios, este historial se almacena en MongoDB.

Prioridad: Alta

Criterio de aceptación: GET /api/mascotas/{id}/historial retorna HTTP 200 con todos los turnos de la mascota ordenados por fecha descendente.

3.5 Autenticación y Autorización

RF-14 — Login de usuario

Descripción: El sistema debe permitir autenticarse con email y contraseña. Ante credenciales válidas, debe retornar un token JWT con el rol del usuario.

Prioridad: Alta

Criterio de aceptación: POST /api/auth/login con credenciales válidas retorna HTTP 200 con token JWT. Con credenciales inválidas retorna HTTP 401.

RF-15 — Control de acceso por rol

Descripción: El sistema debe restringir el acceso a los endpoints según el rol del usuario autenticado. El API Gateway debe validar el token antes de rutear la petición.

Prioridad: Alta

Criterio de aceptación: Una petición a /api/duenos sin token retorna HTTP 401. Un usuario con rol VETERINARIO que intenta acceder a endpoints de administración retorna HTTP 403.

3.6 Resiliencia

RF-16 — Tolerancia a fallos con Circuit Breaker

Descripción: El sistema debe implementar un Circuit Breaker en el API Gateway para las rutas de ms-veterinario. Si el servicio falla más del 50% de las llamadas en una ventana de 10 peticiones, el circuito debe abrirse y devolver una respuesta de fallback.

Prioridad: Alta

Criterio de aceptación: Cuando ms-veterinario está caído, el Gateway retorna HTTP 503 con mensaje descriptivo en lugar de propagar el error. El circuito pasa por los estados CLOSED → OPEN → HALF-OPEN → CLOSED de manera automática.

4. Requisitos No Funcionales — IEEE 29148

Los requisitos no funcionales definen las restricciones de calidad del sistema. Cada requisito incluye una métrica o mecanismo de verificación concreto.

4.1 Rendimiento

RNF-01 — Tiempo de respuesta

Descripción: Los endpoints del sistema deben responder en menos de 500ms para el 95% de las peticiones bajo carga normal (hasta 10 usuarios simultáneos).

Métrica / Verificación: Verificado con Postman / JMeter: p95 < 500ms en pruebas de carga básica.

Prioridad: Alta

RNF-02 — Caché de consultas frecuentes

Descripción: La consulta de veterinarios disponibles debe estar cacheada con Redis con un TTL de 60 segundos para reducir la carga en la base de datos.

Métrica / Verificación: Los logs de ms-veterinario no deben mostrar consultas SQL repetidas dentro del TTL. Verificable via /actuador/metrics.

Prioridad: Alta

4.2 Disponibilidad y Resiliencia

RNF-03 — Aislamiento de fallos

Descripción: Un fallo en cualquier microservicio no debe tumbar al sistema completo. El API Gateway debe continuar respondiendo con fallbacks cuando un microservicio esté caído.

Métrica / Verificación: Con ms-veterinario detenido, el Gateway retorna HTTP 503 con mensaje en menos de 3 segundos. Los demás endpoints del sistema responden normalmente.

Prioridad: Alta

RNF-04 — Health checks

Descripción: Cada microservicio debe exponer un endpoint de salud en /actuator/health que Eureka pueda consultar para determinar la disponibilidad del servicio.

Métrica / Verificación: El dashboard de Eureka muestra el estado UP/DOWN de cada instancia. La transición de DOWN a UP se refleja en menos de 30 segundos.

Prioridad: Alta

4.3 Seguridad

RNF-05 — Autenticación con JWT

Descripción: Todos los endpoints protegidos deben requerir un token JWT válido en el header Authorization. El token debe tener una expiración de 24 horas.

Métrica / Verificación: Una petición sin token o con token expirado retorna HTTP 401. Una petición con token de rol incorrecto retorna HTTP 403.

Prioridad: Alta

RNF-06 — No exposición de credenciales

Descripción: Ninguna credencial (contraseña de base de datos, clave JWT, URL de servicios) debe estar hardcodeada en el código fuente. Deben externalizarse via Config Server o variables de entorno.

Métrica / Verificación: El repositorio no contiene valores sensibles en application.yml. El Config Server provee las propiedades en runtime. Verificable con grep en el repo.

Prioridad: Alta

4.4 Mantenibilidad

RNF-07 — Arquitectura hexagonal en microservicios

Descripción: Cada microservicio de la fase 2 debe seguir la arquitectura hexagonal con separación explícita de capas: domain, application, infrastructure.

Métrica / Verificación: La estructura de paquetes de cada microservicio refleja las tres capas. Las clases de dominio no tienen dependencias de Spring ni de JPA.

Prioridad: Alta

RNF-08 — Cobertura de tests

Descripción: La capa de servicios (Service) de cada módulo debe tener cobertura de tests unitarios con JUnit 5 + Mockito. Al menos el 70% de los métodos públicos deben tener test.

Métrica / Verificación: Los tests pasan en CI (GitHub Actions o ejecución local). Los tests unitarios no levantan el contexto de Spring (`@ExtendWith(MockitoExtension.class)`).

Prioridad: Alta

RNF-09 — Documentación de API

Descripción: Todos los endpoints del sistema deben estar documentados con Swagger/OpenAPI 3. La documentación debe estar disponible en `/swagger-ui.html` en el monolito y en el API Gateway en la fase 2.

Métrica / Verificación: GET `/swagger-ui.html` retorna HTTP 200 con la interfaz de Swagger. Todos los endpoints aparecen con su descripción, parámetros y respuestas posibles.

Prioridad: Media

4.5 Portabilidad y Despliegue

RNF-10 — Contenerización con Docker

Descripción: Cada microservicio debe tener un Dockerfile funcional. El archivo `docker-compose.yml` debe levantar el stack completo (todos los microservicios, MySQL, MongoDB, Redis, RabbitMQ) con un único comando.

Métrica / Verificación: `docker-compose up` levanta el stack completo sin errores. Los microservicios se registran en Eureka en menos de 60 segundos post-inicio.

Prioridad: Alta

RNF-11 — Configuración externalizada

Descripción: Ningún microservicio debe tener configuración de entorno hardcodeada. Toda configuración debe obtenerse del Config Server al iniciar.

Métrica / Verificación: Los microservicios inician correctamente consumiendo configuración del Config Server. Un cambio en el repositorio Git de configuración se refleja en el servicio sin recompilación.

Prioridad: Alta

5. Tabla Resumen de Requisitos

5.1 Requisitos Funcionales

ID	Nombre	Prioridad	Fase
RF-01	Registrar dueño	Alta	1 y 2
RF-02	Consultar dueño	Alta	1 y 2
RF-03	Actualizar dueño	Media	1 y 2
RF-04	Eliminar dueño	Media	1 y 2
RF-05	Registrar mascota	Alta	1 y 2
RF-06	Consultar mascotas de un dueño	Alta	1 y 2
RF-07	Actualizar mascota	Media	1 y 2
RF-08	Registrar veterinario	Alta	1 y 2
RF-09	Consultar veterinarios disponibles (con caché)	Alta	2
RF-10	Registrar turno	Alta	1 y 2
RF-11	Consultar turnos por veterinario	Alta	1 y 2
RF-12	Actualizar estado de turno	Alta	1 y 2
RF-13	Consultar historial de mascotas (MongoDB)	Alta	2
RF-14	Login con JWT	Alta	2
RF-15	Control de acceso por rol	Alta	2
RF-16	Circuit Breaker en Gateway	Alta	2

5.2 Requisitos No Funcionales

ID	Nombre	Categoría	Prioridad
RNF-01	Tiempo de respuesta < 500ms	Rendimiento	Alta
RNF-02	Caché Redis con TTL 60s	Rendimiento	Alta
RNF-03	Aislamiento de fallos (Circuit Breaker)	Disponibilidad	Alta
RNF-04	Health checks con Actuator + Eureka	Disponibilidad	Alta
RNF-05	Autenticación JWT 24h	Seguridad	Alta
RNF-06	Sin credenciales en código fuente	Seguridad	Alta
RNF-07	Arquitectura hexagonal en microservicios	Mantenibilidad	Alta
RNF-08	Cobertura tests 70% en Services	Mantenibilidad	Alta
RNF-09	Documentación Swagger/OpenAPI 3	Mantenibilidad	Media
RNF-10	Docker Compose levanta stack completo	Despliegue	Alta
RNF-11	Configuración externalizada via Config Server	Despliegue	Alta

6. Glosario

Término	Definición
Bounded Context	Límite conceptual dentro del cual un modelo de dominio es coherente y consistente (DDD).
Circuit Breaker	Patrón de resiliencia que interrumpe llamadas a un servicio fallido para evitar propagación de errores.
Config Server	Servidor centralizado que provee configuración a todos los microservicios desde un repositorio Git.
DTO	Data Transfer Object — objeto que transporta datos entre capas sin exponer el modelo de dominio.
Eureka	Servidor de registro y descubrimiento de servicios de Spring Cloud Netflix.

Término	Definición
Feign	Cliente HTTP declarativo de Spring Cloud para comunicación síncrona entre microservicios.
JWT	JSON Web Token — estándar para autenticación stateless mediante tokens firmados.
Microservicio	Servicio independiente con responsabilidad única, base de datos propia y ciclo de deploy propio.
Monolito MVC	Arquitectura donde toda la lógica reside en una única aplicación desplegable.
Redis	Base de datos en memoria usada como caché de alta velocidad.
REST	Estilo arquitectural para APIs HTTP basado en recursos, verbos y representaciones.
Sprint	Iteración de tiempo fijo (en esta materia: una clase) con un objetivo y entregable concreto.